

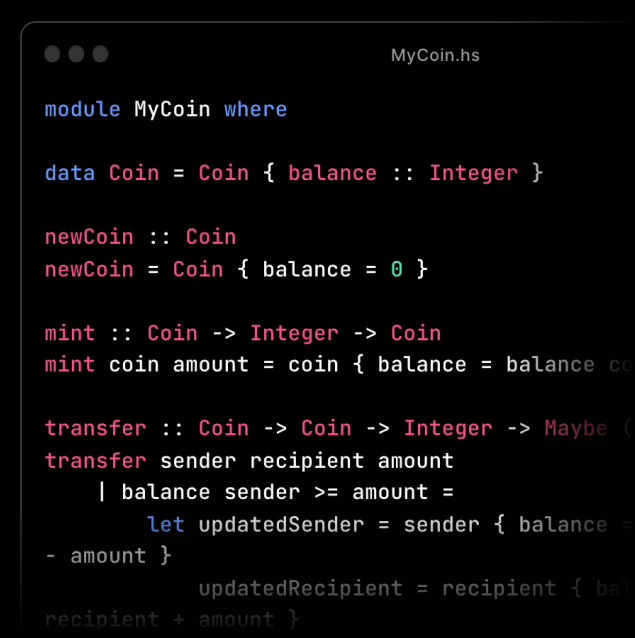
Programação Funcional

Prática a01

HASKELL

Linguagem funcional criada por um comitê de investigadores em **1990**, com o nome inspirado no matemático **Haskell Curry**.

- Uma linguagem de programação **funcional**.
- É uma linguagem pura — **evita efeitos secundários** no código funcional.
- Usa avaliação lazy — calcula expressões **quando o resultado é necessário**.
- Tem **tipagem** estática e forte **inferência de tipos**.



```
module MyCoin where

data Coin = Coin { balance :: Integer }

newCoin :: Coin
newCoin = Coin { balance = 0 }

mint :: Coin -> Integer -> Coin
mint coin amount = coin { balance = balance coin + amount }

transfer :: Coin -> Coin -> Integer -> Maybe ()
transfer sender recipient amount
  | balance sender >= amount =
    let updatedSender = sender { balance = balance sender - amount }
    in updatedRecipient = recipient { balance = balance recipient + amount }
  | otherwise = Nothing
```

<https://www.codezeros.com>

AVALIAÇÃO

50% Teórica + 50% Prática

- 2 frequências (10 de novembro e 5 de janeiro)
- Projeto (**com vários checkpoints ao longo do semestre**)

! Nota mínima 7 valores em cada componente

! Aulas práticas não são obrigatórias

SETUP

<https://github.com/v2denny/pf2627>

<https://www.haskell.org/ghcup/install/>



GRUPOS

Grupos para o projeto prático (50% da nota)

- 3 por grupo
- tentar sentar em grupo

TESTAR INSTALAÇÃO

```
(base) C:\Users\danie\Desktop\AULAS\pf2627\aulas\01>ghc --version
The Glorious Glasgow Haskell Compilation System, version 9.10.3

(base) C:\Users\danie\Desktop\AULAS\pf2627\aulas\01>ghci
GHCi, version 9.10.3: https://www.haskell.org/ghc/  :? for help
ghci> 1+2
3
ghci> 3>4
False
ghci> :type 'a'
'a' :: Char
ghci> :t "abc"
"abc" :: String
ghci> :q
Leaving GHCi.
```

CÓDIGO a01

```
ghci> :load Main.hs
Ok, one module loaded.
ghci> Aluno "Ana" 15
Aluno {nome = "Ana", nota = 15.0}
ghci> a = Aluno "Ana" 15
ghci> nome(Aluno "Ana" 15)
"Ana"
ghci> nome(a)
"Ana"
ghci> nota(a)
15.0
```

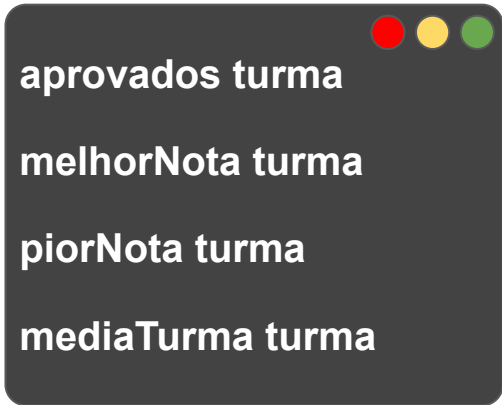
1. Iniciar o GHCi e carregar o programa
2. Criar manualmente um aluno
3. Ver o nome e nota do aluno

CÓDIGO a01

Criar uma turma de teste

```
let turma = [Aluno "Ana" 15, Aluno "Bruno" 8, Aluno "Carlos" 12, Aluno "Diana" 18]
```

Testar aprovados:



```
aprovados turma
```

Testar melhorNota:

```
melhorNota turma
```

Testar piorNota:

```
piorNota turma
```

Testar mediaTurma:

```
mediaTurma turma
```

CÓDIGO a01

Testar a leitura do ficheiro "alunos.txt"

```
readFile "alunos.txt"
conteudo <- readFile "alunos.txt"
lines conteudo
let alunos = map parseAluno (lines conteudo)
aprovados alunos
melhorNota alunos
piorNota alunos
mediaTurma alunos
```

CÓDIGO a01

```
ghci> :q
Leaving GHCi.

(base) C:\Users\danie\Desktop\AULAS\pf2627\aulas\a01>ghc Main.hs -o turma

(base) C:\Users\danie\Desktop\AULAS\pf2627\aulas\a01>turma.exe
=== RESULTADOS DA TURMA ===

Alunos aprovados:
Ana - 15.0
Carlos - 12.0
Diana - 18.0
Filipa - 16.0
Helena - 14.0

Estatísticas:
Melhor nota: 18.0
Pior nota: 7.0
Media: 12.5
```

Compilar o projeto e executar

- Windows: turma.exe
- Linux/MacOS: ./turma